

Machine Learning Lecture Study & Reference Notes

Based on: "Notes from Lecture(1).pptx"

These notes reorganize the lecture into a study-friendly sequence. The PPT is largely handwritten/image-based, so a few topics are listed only briefly. Where the lecture gives only a heading or example, that limitation is explicitly noted rather than pretending the missing material was in the slides.

1. Lecture Roadmap

The lecture touches on **feature engineering**, **SVM**, **model evaluation**, **decision trees**, **KNN**, **ensemble learning**, and **hyperparameter tuning**. The opening notes also list PCA (Principal Component Analysis).

Area	Main idea	Examples in lecture
Supervised learning	Learn from labelled input/output data.	Regression, classification, SVM, KNN, Decision Tree
Unsupervised learning	Find structure in data without a target label.	Clustering, PCA
Regression	Predict a continuous numeric value.	Linear Regression, SVM regression, KNN regression
Classification	Predict a category/class.	Logistic Regression, SVM classification, KNN classification, Decision

2. Feature Engineering

Definition: Feature engineering is the process of working with the columns/features of a dataset so that the data becomes more suitable for a machine-learning model.

Lecture workflow: Outliers → Null values → Encoding → Feature scaling → Feature selection.

2.1 Outliers and Null Values

Outlier: an observation unusually far from the rest of the data. Outliers can strongly affect models and statistics. Before removing one, check whether it is a genuine observation or an error.

Null value: a missing value. Typical handling includes removing rows/columns when justified or imputing values (for example, median for a numeric feature). The PPT identifies outliers and null values as feature-engineering concerns, but does not give a detailed imputation procedure.

2.2 Encoding Categorical Variables

Why? Most ML algorithms need numerical input, while real datasets often contain text such as Yes/No, Male/Female, or Small/Medium/Large.

Method	Idea	Example
One-hot encoding	Create one binary column for each category.	Color = red/green/blue → red, green, blue columns with 0/1
Label encoding	Map categories to integer labels.	Yes → 1, No → 0
Ordinal encoding	Map categories according to their meaningful order.	Small → 0, Medium → 1, Large → 2

Important distinction: One-hot encoding does not imply an order. Ordinal encoding should be used when the categories genuinely have an order. For binary categories such as Yes/No, a 0/1 representation is usually sufficient.

2.3 Feature Scaling

Scaling puts numeric features on comparable ranges. This is especially important for distance- or geometry-based methods such as KNN and SVM because a large-scale feature can otherwise dominate the calculation.

Normalization (Min-Max Scaling)

$$x_{\text{scaled}} = (x - x_{\min}) / (x_{\max} - x_{\min})$$

This commonly maps values into the range [0, 1]. Lecture example: if min = 0, max = 15 and x = 2, then $2/15 \approx 0.13$.

Standardization (Standard Scaler)

$$z = (x - \mu) / \sigma$$

μ = mean, σ = standard deviation. After standardization, the transformed feature has mean approximately 0 and standard deviation approximately 1.

Quick choice: Min-Max scaling gives a bounded range; standardization expresses values in standard-deviation units. The best choice depends on the model and data.

2.4 Feature Selection

Feature selection means choosing the useful input features and dropping unnecessary/redundant ones. The lecture uses correlation as the main idea.

Correlation with the target: if a feature and target move together, the relationship may be positive or negative. A positive relationship means they tend to increase together; a negative relationship means one tends to increase while the other decreases. A correlation near zero indicates little linear relationship.

Correlation between input features: if two predictors carry very similar information, one may be removed to reduce redundancy, depending on the model and the goal.

Lecture example: if x_1 and x_2 are highly correlated, dropping one can reduce redundant information. Feature selection is not the same thing as feature extraction: selection keeps existing columns, while extraction creates new representations.

3. Support Vector Machine (SVM)

The lecture describes SVM as a **supervised machine-learning algorithm** that can solve both **classification** and **regression**. Its central idea is to find a good separating boundary/hyperplane.

3.1 Hyperplane / Decision Boundary

For classification, a decision boundary separates classes. In two dimensions it can be a line; in higher dimensions it is called a hyperplane.

3.2 Support Vectors

Support vectors are the training points closest to the decision boundary/hyperplane. They are the critical observations that determine the position of the maximum-margin boundary.

Memory trick: Support vectors = the points supporting/defining the boundary.

3.3 Margin

The margin is the distance between the decision boundary and the closest points from the classes. SVM seeks a boundary with a large margin, which is the basis of the “maximum-margin” idea shown in the lecture.

As the candidate line changes, the closest points can change. The lecture explicitly notes that support vectors can change when the line changes.

3.4 Linear vs Non-linear Separation

If a straight line/hyperplane can separate the classes, the problem is linearly separable. If it cannot, the data is non-linearly separable and SVM can use a **kernel**.

3.5 Kernel Trick

The lecture lists **linear**, **polynomial (poly)**, and **RBF** kernels. The purpose is to make a non-linear separation possible by representing the data in a transformed/higher-dimensional feature space.

Lecture example: one-dimensional data can be transformed into a higher-dimensional representation such as $x \rightarrow x^2$. The transformed representation can make a pattern easier to separate with a linear boundary.

Kernel intuition: instead of manually creating every higher-dimensional feature, a kernel lets the algorithm work with relationships corresponding to that transformed space.

3.6 SVM for Regression

SVM is not limited to classification. In regression, the objective is to model a continuous target while controlling the fit using the SVM regression formulation.

3.7 SVM Study Checklist

Term	Remember
Hyperplane	The separating decision surface.
Support vectors	Closest/critical training points that define the margin.
Margin	Distance around the boundary to the closest points.
Kernel	Technique used to handle non-linear relationships through transformed feature space.
Linear kernel	Useful when a linear boundary is adequate.
Polynomial kernel	Can model polynomial-type non-linear relationships.
RBF kernel	Common non-linear kernel based on similarity around points.

4. Model Evaluation

The lecture says evaluation metrics are used **to check the performance of the model**.

4.1 Regression Metrics

R² (R-squared): measures how much of the variation in the target is explained by the model, relative to a baseline. Higher is generally better; 1 indicates a perfect fit, while values can also be negative for a model performing worse than the baseline.

MSE (Mean Squared Error): average of squared prediction errors. Larger errors are penalized more heavily.

$$\text{MSE} = (1/n) \sum (y - \hat{y})^2$$

RMSE: square root of MSE. It is in the same units as the target, which often makes it easier to interpret.

$$\text{RMSE} = \sqrt{\text{MSE}}$$

MAE (Mean Absolute Error): average absolute prediction error.

$$\text{MAE} = (1/n) \sum |y - \hat{y}|$$

4.2 Classification Metrics

The lecture lists **accuracy, precision, recall, F1 score, ROC-AUC** and **confusion matrix**.

Confusion matrix: summarizes predicted vs actual classes using True Positive (TP), True Negative (TN), False Positive (FP), and False Negative (FN).

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$$

$$\text{Precision} = TP / (TP + FP)$$

$$\text{Recall} = TP / (TP + FN)$$

$$F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

When to care: accuracy is simple but can be misleading with imbalanced classes. Precision matters when false positives are costly; recall matters when false negatives are costly. F1 balances precision and recall. ROC-AUC evaluates ranking/separation across thresholds.

5. Decision Tree

The lecture describes a decision tree as a supervised ML algorithm that can solve **classification and regression**. It repeatedly splits the data into branches using feature-based decisions.

5.1 Tree Structure

Root node: starting point. **Internal node**: a decision/split. **Branch**: outcome of a split. **Leaf node**: final prediction.

5.2 Splitting Criteria

The lecture lists **Information Gain (ID3)**, **Gini Index**, and **Entropy**.

Entropy: measures impurity/disorder in a node. For class probabilities p_i :

$$\text{Entropy} = - \sum p_i \log_2(p_i)$$

Pure node → entropy 0. A highly mixed node has higher entropy. **Information Gain** compares impurity before and after a split; the lecture notes $IG = \text{parent entropy} - \text{weighted child entropy}$, and that a higher information gain is preferred.

Gini Index: another impurity measure used in tree splitting, commonly associated with CART. Lower impurity is preferred.

5.3 Overfitting and Pruning

A very deep tree can memorize the training data and perform poorly on unseen data. This is overfitting. **Pruning** reduces unnecessary branches/tree complexity to improve generalization.

6. K-Nearest Neighbours (KNN)

The lecture describes KNN as a supervised ML algorithm that can solve both regression and classification. Its prediction is based on nearby training observations.

6.1 Core Idea

K = number of neighbours considered. A new data point is compared with training points, the nearest K are selected, and their values/classes are used for prediction.

Because KNN depends on distance, **feature scaling is particularly important**. Otherwise, a large-range feature can dominate the distance calculation.

6.2 Distance Measures

Euclidean distance (2D): $d = \sqrt{(x_{\text{new}} - x_{\text{train}})^2 + (y_{\text{new}} - y_{\text{train}})^2}$

Manhattan distance: $d = |x_{\text{new}} - x_{\text{train}}| + |y_{\text{new}} - y_{\text{train}}|$

The lecture explicitly demonstrates Euclidean and Manhattan distance and then discusses choosing K.

6.3 Choosing K

The lecture notes: if K is too small, the model is sensitive to noise/high variance; if K is too large, the model can become too smooth and make poor local decisions. A common classroom rule shown is $K = \sqrt{N}$, where N is the number of training samples, but this should be treated as a rough starting point rather than a universal optimum.

Default shown in the lecture: K = 5.

6.4 Classification vs Regression

Classification: use the classes of the nearest neighbours, typically by majority vote. **Regression:** combine the neighbours' numeric target values, commonly using their mean.

7. Ensemble Learning

The lecture defines ensemble learning as an ML technique where multiple models are combined to produce a stronger overall model.

The two techniques explicitly listed are **Bagging** and **Boosting**.

7.1 Bagging

Bagging trains multiple models independently, typically using different bootstrap samples of the training data, and combines their predictions. Because the models can be trained in parallel, the lecture emphasizes parallel training.

Example idea: train several trees → collect predictions → combine them (majority vote for classification or averaging for regression).

7.2 Boosting

Boosting builds models sequentially. Later models focus more on observations that previous models handled poorly, so the ensemble gradually improves the overall predictor.

Key contrast: Bagging = independent/parallel models; Boosting = sequential models that build on earlier errors.

8. Hyperparameter Tuning

The lecture defines hyperparameters as values set for the model before/during training rather than learned directly from the training data. Hyperparameter tuning means finding good values for them to improve model performance.

Examples from the lecture: SVM, Decision Tree, KNN and ensemble methods have tunable settings. For KNN, K is a key hyperparameter. For a tree, settings such as depth/complexity affect overfitting. For SVM, kernel choice and parameters such as C/gamma can matter.

Important distinction: Parameters are learned from training data; hyperparameters are chosen/set by the practitioner or a search procedure.

9. PCA: What the PPT Actually Covers

PCA (Principal Component Analysis) is listed in the opening lecture notes, but the provided PPT does not contain a substantive step-by-step explanation of PCA. Therefore, the detailed PCA theory below is a reference addition, not a reconstruction of missing lecture content.

Reference idea: PCA is a dimensionality-reduction technique that transforms correlated features into a smaller number of uncorrelated principal components that capture as much variance as possible.

Typical conceptual steps: center/standardize data → compute covariance structure → find principal directions → rank components by explained variance → keep the desired number of components → transform the data.

10. High-Yield Comparison Table

Algorithm/Concept	Type	Core idea	Important point
Linear Regression	Supervised / Regression	Fit a relationship for a continuous target.	Predicts numeric values.
Logistic Regression	Supervised / Classification	Predict class probabilities using a logistic model.	Classification despite the word 'regression'.
SVM	Supervised / Both	Maximum-margin boundary; kernels handle non-linear patterns.	Support vectors + margin.
KNN	Supervised / Both	Predict from nearby training points.	Distance + K; scaling matters.
Decision Tree	Supervised / Both	Recursive feature-based splits.	Entropy/Gini; prune to control overfitting.
Ensemble	Supervised technique	Combine multiple models.	Bagging vs Boosting.
PCA	Unsupervised / Dimensionality reduction	Transform features into principal components.	Reduce dimensions while preserving variance.

11. Exam / Interview Quick Revision

Feature Engineering: clean → encode → scale → select useful features.

One-hot: unordered categories → separate 0/1 columns.

Ordinal: ordered categories → numbers preserving order.

Normalization: $(x - \min) / (\max - \min)$, commonly $[0, 1]$.

Standardization: $(x - \text{mean}) / \text{SD}$, mean ≈ 0 and SD ≈ 1 .

SVM: find a maximum-margin boundary; closest points are support vectors.

Kernel: helps SVM handle non-linear separation.

Decision Tree: split data recursively; entropy/Gini measure impurity; pruning controls overfitting.

KNN: nearest K points decide the prediction; distance and scaling matter.

Bagging: independent models, often parallel, aggregate predictions.

Boosting: sequential models, later models focus on previous errors.

Evaluation: regression → R^2 , MAE, MSE, RMSE; classification → confusion matrix, accuracy, precision, recall, F1, ROC-AUC.

Hyperparameter tuning: search for useful settings such as K, tree depth, kernel parameters, etc.

12. Common Mistakes to Avoid

1. Treating every categorical column as ordinal. Only ordered categories should receive an order-based encoding.
2. Forgetting to scale before distance-sensitive algorithms such as KNN.
3. Thinking every high correlation automatically means a feature must be removed. Context, model type, leakage, and domain meaning matter.
4. Confusing support vectors with all training points. Only the critical points near the boundary are support vectors.
5. Thinking a deep decision tree is automatically better. It can overfit.
6. Assuming $K = \sqrt{N}$ is always optimal. It is only a heuristic; validation should guide the final choice.
7. Using accuracy alone on an imbalanced classification problem.

13. One-Page Mental Map

Raw data



Feature Engineering → handle outliers/nulls → encode categories → scale numeric features → select useful features



Model → SVM / KNN / Decision Tree / Regression / Ensemble



Train → tune hyperparameters using validation



Evaluate → regression metrics OR classification metrics



Improve → better features + suitable model + tuning + control overfitting

Source note: The supplied lecture file explicitly identifies feature engineering and the ML topics, and its SVM section explicitly covers support vectors and changing support vectors as the boundary changes. The rest of this document organizes the concepts visible in the handwritten slides and adds small explanatory bridges where the slides are abbreviated.